Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Analyzing algorithms with big O

Today
Welcome to your next lesson: Analyzing algorithms with big O.

I'm setting up your personalized lesson — based on what you've learned so far, what's next, and the best way for you to practice.

Step by step, you’re moving forward.

Imagine two friends sorting trading cards ☺

Alex looks at each card twice: once to read it, once to sort it.
Jamie looks at each card once, but also adds 5 stickers at the end for fun.

For a huge collection (say 10,000 cards), both are basically doing “go through all the cards” — the extra “once more” or “+5 stickers” doesn’t really change the overall work. In Big O, we say both are O(n) and we ignore those small constant extras like “×2” or “+5”.

So in Big O, we care about the dominant growth: the part that grows fastest as n gets big, and we drop:

constant multipliers (like 2 * n)
small extra chunks (like + 5 or + 100)

Make sense so far, in your own words?

Is it clear why 2 * n + 5 and n are considered the same Big O class (O(n))?

yes makes sense

Nice 👍

Let’s connect that “ignore the small stuff” idea to a tiny Python example.

Look at this fake “step count” for some function:

Steps(n) = 2 prints before the loop

n prints inside a loop

= n + 2 total prints

For Big O:

The n part grows as input grows
The + 2 is just a tiny constant extra
So we say this is O(n) and we ignore the + 2.
Quick check in your head:

If we change it to n + 100 steps instead of n + 2, what's the Big O class: O(1), O(n), or O(n²)?

You can just answer with the label (like: O(n)).

o(n)

Exactly — O(n) ☑

We don't care if it's n + 2 or n + 100 or 5n + 7 — all of those grow linearly with n, so they're all O(n). The big picture is "if n doubles, steps roughly double."

Next, let's look at a concrete Python snippet and classify it.

Python

```
def example(n):
    print("start")
    print("setup done")
    for i in range(n):
        print(i)
```

Step-by-step:

2 prints before the loop → constant work
1 print per loop, repeated n times → grows with n

So total steps ≈ n + 2, which we call O(n).

Ready for a quick comparison?

Which one do you expect to be O(1) (constant time)?

A) A function that always does exactly 5 prints, no loops.

B) A function that loops from 0 to n and prints each number once.

Just answer A or B.

a

Nice, A is right 🎯

A: always 5 prints → doesn't grow with n → O(1)
B: n prints → grows with n → O(n)
Now I want to bring in worst case vs best case — still with simple ideas.

Imagine this function that searches for a number in a list:

Python

```
def find_first(nums, target):
```

```
    for x in nums:
        if x == target:
            return True
    return False
```

Best case: target is the first element → we stop after 1 check
Worst case: target is not in the list (or last) → we check all elements

In Big O, when someone says “this is O(n)”, they usually mean the worst case: we might have to look at every element.

Does that idea land: Big O normally talks about the worst situation, not the lucky easy one?

Yes, that makes sense

Perfect ✸

Let’s zoom in on one concrete piece now: spotting O(1), O(n), O(n²) from tiny Python snippets.

I’ll show you one snippet and we’ll classify it together.

Python

```
def demo(n):
    print("hello")
    for i in range(n):
        print(i)
    for j in range(n):
        print(j)
```

Step-by-step:

print("hello") → 1 step (constant)
First loop → runs n times → about n steps
Second loop → also runs n times → about n steps

Total steps ≈ 1 + n + n = 2n + 1

For Big O, we drop constants, so 2n + 1 behaves like just n → this is O(n).

Quick check:

If I changed it so there were 3 separate loops, each from 0 to n, what Big O class would it be: O(1), O(n), or O(n²)?

Just answer with the class (like O(n)).

o(n)

Exactly, still O(n) ☑

Three loops from 0 to n is about 3n steps → drop the 3 → linear.

Now let’s contrast that with a nested loop, which gives us O(n²).

Python

```
def demo_square(n):
    for i in range(n):
        for j in range(n):
            print(i, j)
```

Step-by-step:

Outer loop runs n times

For each i, inner loop runs n times
Total prints ≈ n * n = n² → that's O(n²)

Visual idea:

Single loop → steps grow like: n
Double nested loop → steps grow like: n × n = n²
I want to check this idea with you using a small thought experiment:

If n = 3, how many times does print(i, j) run in demo_square?

Just give the number (like 9, 6, etc.).

9

Exactly, 9 is right 🎯

For n = 3, the pairs are:

i = 0 → j = 0,1,2 → 3 times
i = 1 → j = 0,1,2 → 3 times
i = 2 → j = 0,1,2 → 3 times

Total = 3 + 3 + 3 = 9 = 3² → matches the n² pattern.

So now you've seen:

O(1) → fixed work, no loop depending on n
O(n) → one (or a few) loops that each go up to n
O(n²) → nested loops n × n

I want to give you a tiny "spot the complexity" challenge next.

Ready to try a quick multiple-choice question on that?

Yes, give me the challenge

What do you think the answer is? Take your time with the question above 👆

What is the Big O time complexity of this function?

```
def mystery(n):
    print("start")
    for i in range(n):
        for j in range(n):
            print(i, j)
    print("end")
```

Remember: ignore constants like the extra prints before/after the loop.

A.

O(1)

B.

O(n)

C.

O(n²)

Nice work, Ron — that was spot on 🎉

mystery(n) has:

1 print before the loops → constant work
1 print after → constant work
A nested loop: outer loop runs n times, and for each i, inner loop also runs n times → about n * n = n² prints

So the total work looks like: n² + (small constants), and when we ignore constants and smaller terms, the dominant behavior is n², so the time complexity is O(n²).

Now I want to check one more key idea with you: worst case vs best case using find_first:

Python

```
def find_first(nums, target):
    for x in nums:
        if x == target:
            return True
    return False
```

Think about a list of n items:

Best case: target is the first element → we check 1 item, then stop
Worst case: target is not in the list → we check all n items

In Big O language, when someone says "find_first is O(n)", they usually mean the worst case: we might have to scan the whole list.

To check your understanding:

Is the best case for find_first closer to O(1) or O(n)? Just answer O(1) or O(n).

o(n)

Nice thinking — I see why you picked O(n), because in the worst case we do look through all n items.

But here we're talking best case only: if the target is the first element, we check just 1 item and stop. That's constant work, no matter how big n is → best case is O(1). Worst case is O(n).

So for find_first:

Case	Steps checked	Big O
Best	1	O(1)
Worst	n	O(n)

And when people say "this algorithm is O(n)", they almost always mean the worst case.

Want to try a quick multiple-choice question to lock in this "worst vs best case" idea?

Yes, give me a question

What do you think the answer is? Take your time with the question above 👇

You use this function to check if a number is in a list:

```
def contains(nums, target):
    for x in nums:
        if x == target:
            return True
    return False
```

What is the worst-case time complexity (in Big O) for contains when the list has n elements?

A.

O(1)

B.

O(n)

C.

O(n²)

Nice job, Ron — you nailed it again 🎯

For contains:

Python

```
def contains(nums, target):
    for x in nums:
        if x == target:
            return True
    return False
```

In the worst case, the target is not in the list (or at the very end).
That means we check all n elements one by one.
Work grows linearly with n, so the worst-case time is O(n).

Best case would be O(1) (if the first element matches), but Big O usually talks about the worst case, which you correctly picked. 👍

You've now used:

Ignore constants → focus on dominant term
Spotting O(1), O(n), O(n²) from code
Understanding worst vs best case in Big O

From your answers, you've mastered everything in this lesson. 🎓

Are you ready to wrap up this lesson and move on to the next one, or do you want one more practice question first?

Move to the next lesson

Awesome work today, Ron 🎯

Quick recap of what you now own:

You can ignore constants and lower-order terms and focus on the dominant part (like turning 2n + 10 into just O(n)).
You can spot O(1), O(n), and O(n²) in small Python snippets by looking at how loops are structured (no loop, single loop, nested loops).
You understand that Big O usually means worst-case — like contains being O(n) because sometimes we must scan the whole list, even though best case is O(1).
You're ready for the next step in data structures & algorithms 🚀
When you're ready, hit Next Lesson in the Learn tab, and I'll be there to keep going with you.